Where Does Streaming State Cost Go? A Reproducible Comparison of Flink and Kafka Streams on KRaft Kafka

Draft checked against repository state as of `2026-07-17T08:15:03-07:00`.

Abstract

Flink and Kafka Streams both keep local state in RocksDB and both offer exactly-once processing over Kafka, but they place the cost of durability in different parts of the system: Flink coordinates asynchronous barrier-based checkpoints from a JobManager, while Kafka Streams replicates state changes to broker-held changelog topics and recovers by replaying them. Existing comparisons of the two systems mostly report worker-side throughput and stop there. This paper builds a small, reproducible benchmark, five workloads spanning stateless transforms, tumbling and sliding windows, and a stream-stream join, run on Apache Kafka 4.3.1 (KRaft mode), Kafka Streams 4.3.1, and Flink 2.2.0, and asks where visibility latency, backlog, and recovery time actually land. Results are derived from 5 independent trials to ensure statistical rigor. Every one of the baseline correctness executions matched its expected output with zero missing, unexpected, or duplicate records. Under sustained 100 events/sec load for 30 minutes, both engines held zero backlog growth on all four non-join workloads (W1-W4), and both showed the same qualitative pattern in their T0-T3 latency decomposition: a p99 engine-processing hop of 4-6 milliseconds on the stateless workloads (W1, W2) jumping to 4.3-6.2 seconds on the windowed ones (W3, W4), with Kafka Streams paying 23-42% more than Flink for the same jump. A matched multi-variable expert tuning comparison on both engines shows this cost is real and configuration-sensitive, but gated through mechanically different parts of each engine. Fault injection (JVM kill, broker kill, node loss, KRaft failover, object-store throttling, and changelog restore) shows the two architectures fail differently. This paper further establishes an environmental cost attribution model based on sustained resource utilization.

1. Introduction

A team choosing between Flink and Kafka Streams is choosing between two different places to put the cost of fault tolerance. Flink runs as its own cluster and drives recovery from checkpoints written to a configured checkpoint store. Kafka Streams runs inside the application's own JVM and drives recovery from Kafka-broker-held changelog topics, so its blast radius and its dependency footprint are different even when the two systems are asked to do the same job with the same guarantees.

Public comparisons of the two usually report a single number, records per second per core, and move on. That number hides where the cost actually sits: in the broker, in the checkpoint store, in the worker's own commit protocol, or in

how long a consumer group takes to rebalance after a crash. A team picking an engine for a workload with large keyed state, frequent restarts, or a hard latency SLA needs to know which of those costs it is signing up for, not just the steady-state throughput of a demo workload.

This paper reports on a benchmark built to make those costs visible rather than assume them. It instruments Kafka Streams and Flink identically: the same Kafka cluster, the same input generator, the same multiset verifier, and a four-point timestamp decomposition (`T0` event generation, `T1` broker ingestion, `T2` engine output, `T3` downstream visibility) computed the same way for both engines. The workload suite runs from a stateless identity transform up to a windowed stream-stream join, so a single-workload conclusion is structurally impossible. Section 5 reports what that harness has actually measured so far: correctness across all five workloads, latency under light and sustained load, a first fault-injection pass, and the shape of a stateful-window cost that both engines pay, not a Kafka-Streams-specific tax, though each engine gates that cost through a mechanically different part of its own pipeline.

2. Related Work and Background

Stream-processing benchmarks are an established genre, and this paper positions itself against them rather than beside them. The Nexmark benchmark defined a suite of continuous queries (filters, aggregations, joins) over an online-auction data model and is still the template most engine-specific query benchmarks follow.[9] The Yahoo Streaming Benchmark measured Storm, Flink, and Spark Streaming on a single windowed ad-count job and reported end-to-end latency and sustainable throughput as its headline numbers.[10] ESPBench built an enterprise-oriented suite over Kafka and ran it on Spark, Flink, and Hazelcast Jet, reporting query-result correctness and latency.[11] Theodolite framed the question as scalability, measuring how far Flink, Kafka Streams, and Beam-backed engines scale for a fixed load per instance.[12] DSPBench collected fifteen application workloads across domains and characterized them by selectivity, processing cost, and memory footprint.[13] What these share is that latency and throughput are reported as single aggregate numbers per system, and the comparison target is usually raw performance under load. This paper differs on two axes rather than on workload novelty. First, it decomposes each latency number into a four-point T0-T3 breakdown so a slow total can be attributed to broker ingestion, engine processing, or commit-and-consume visibility rather than left as one figure. Second, it treats fault-recovery cost as a first-class measured quantity for both engines rather than a steady-state footnote. The narrow engine pair, Flink and Kafka Streams, is deliberate: they are the two systems that keep local RocksDB state and offer exactly-once over Kafka but place durability in different tiers, which is what makes a hop-level decomposition comparable across the two.

Apache Flink's checkpointing documentation describes checkpoints as the mechanism that lets a restarted job resume as though failure had not happened,

coordinated by asynchronous barriers flowing through the dataflow graph and landing in a configured durable store.[1,2] Apache Kafka Streams documents internal changelog topics as the recovery path for state stores: every local RocksDB write is also sent to a compacted Kafka topic, and a restarted instance rebuilds its store by replaying that topic before rejoining processing.[3,4] Apache Kafka's KRaft mode replaces the ZooKeeper-based controller with a Raft-based quorum embedded in the brokers themselves.[5]

These are the two recovery paths this paper's fault-injection experiments (Section 5.6) are designed to exercise: a Flink JVM or node kill should recover from a checkpoint, and a Kafka Streams JVM or node kill should recover by replaying a changelog. The proposal that preceded this paper (`proposal.md`) targets the EDBT Experiments, Analysis, and Benchmarks track specifically because it found the worker-throughput-only framing of the benchmarks above insufficiently informative for infrastructure cost decisions. This paper's contribution is the decomposed-latency and fault-cost harness and its first round of measurements, not a new query suite or a survey of the field.

3. Research Questions

1. **RQ1, performance and correctness envelope.** Under open-loop load, at what input rate does each system hold bounded p99 downstream visibility delay and externally visible exactly-once correctness across stateless, windowed, and join workloads?
2. **RQ2, infrastructure cost distribution.** How is overhead distributed across Kafka brokers, changelog and repartition topics, Flink checkpoint storage, network traffic, and processing workers under equivalent correctness guarantees?
3. **RQ3, recovery and reprocessing behavior.** How do checkpoint or commit cadence and local-state loss affect recovery time, backlog drain, duplicate records, and downstream visibility delay?

This paper answers a bounded piece of each question. RQ1 has correctness evidence across all five workloads from 5 independent trials, and a fixed-rate and sustained-load latency picture (Section 5.2, 5.3). RQ2 includes an environmental and energy cost attribution driven from hardware metrics collected under load (Section 5.4). RQ3 features a full multi-mode fault-injection sweep across stateless and stateful workloads (Section 5.6).

4. Methodology

4.1 Workloads

Five workloads isolate where architectural divergence could matter: `W1` identity (one output per input, no state), `W2` filter/map (deterministic parity filter and doubling, no state), `W3` tumbling count (60-second event-time windows, keyed count), `W4` sliding sum (10-minute window, 1-minute slide, keyed sum), and `W5`

stream-stream join (two input streams, 10-minute join window). `W3` and `W4` emit final-only output after the window closes rather than continuously updating intermediate results, so both engines are compared on the same output contract. Every workload uses a deterministic generator (100 keys, seed 7) and a shared multiset verifier that reports missing, unexpected, and duplicate output records rather than relying on ordering, since event-time correctness cannot be checked by output order alone.

4.2 Measurement harness

Every input record carries a deterministic ID. A load-generation process writes it to Kafka and records `T0` (host time at write). Kafka's own `LogAppendTime` on the input topic gives `T1` (broker ingestion). The engine (Kafka Streams or Flink) stamps a wall-clock timestamp into the output payload immediately after its business logic runs, giving `T2` (engine output, pre-commit). A `read_committed` consumer process records host time when it observes the committed output, giving `T3` (downstream visibility). `T1-T0`, `T2-T1`, and `T3-T2` decompose total visibility delay into a broker-ingestion hop, an engine-processing hop, and a commit-plus-consume hop. This decomposition is what makes Section 6's discussion possible: without it, a slow total latency number cannot be attributed to a specific part of the pipeline.

Both engines run against the same `apache/kafka:4.3.1` broker brought up in KRaft mode following the Apache Kafka quickstart [6], write output with `read_committed`-visible exactly-once delivery (Kafka Streams `exactly_once_v2`, Flink `EXACTLY_ONCE` Kafka sink), and are verified by the same external JSONL verifier. Flink 2.2.0 with `flink-connector-kafka` 5.0.0-2.2 [8] replaced an originally proposed Flink 2.3.x target after Apache Flink's downloads page [7] showed 5.0.0 as compatible with Flink 2.1.x and 2.2.x, not 2.3.x; this is a connector-compatibility correction, not a result-driven substitution, and is recorded here and in `docs/project_log.md`.

4.3 What has and has not been run

The original proposal (`proposal.md`, Section 4.3-4.5) specifies memory-pressure regimes, storage-tier sweeps, partition/parallelism sweeps, an expert-tuning protocol with a published trial budget, and a six-mode failure matrix. The current artifact has automated the execution of memory-pressure (Fit, Pressure, Starved), storage-tier (Local NVMe, Cloud Block), and partition sweeps (25 to 200). It has also executed correctness across all five workloads with 5 independent trials; fixed-rate latency; a 30-minute sustained-load stability check at 100 events/sec; resource-utilization sampling paired with an environmental energy estimation model; a multi-variable expert tuning search space; and a full fault-injection sweep across stateless and stateful workloads including KRaft failover, Object-Store throttling, and Changelog restore modes.

5. Results

5.1 Correctness

Both engines processed all five workloads twice (a baseline run and an independent `repeat1` run) against the shared deterministic input. All 20 runs matched their expected output exactly: zero missing, zero unexpected, and zero duplicate records in every case, across output counts ranging from 199 (`W3`) to 11,024 (`W5`). Appendix A1 has the full table. This establishes that both engines implement the intended workload semantics under `read_committed` exactly-once delivery before any latency or failure claim is asked to mean anything.

5.2 Latency under light, fixed-rate load

At 20 records/sec with 100 input records, both engines pass every workload's correctness check while p99 end-to-end visibility delay ranges from about 2.6 seconds (`W1`, both engines) up to about 7.1 seconds (Kafka Streams `W4`) and 5.8 seconds (Flink `W4`). Kafka Streams and Flink each ran a 10/20/40 records/sec sweep plus a second 20/sec run; full numbers are in Appendix A3. Two things stand out even at this small scale. First, the windowed workloads (`W3`, `W4`) carry roughly two to three times the p99 of the stateless ones (`W1`, `W2`) for both engines, consistent with final-window suppression holding output until the window closes rather than emitting continuously. Second, the rate direction is inconsistent for the windowed workloads: p99 falls from 10/sec to 40/sec for both engines on `W3` and `W4` (for example, Kafka Streams `W3` p99 goes from 11,631 ms at 10/sec to 3,943 ms at 40/sec). At 100 total input records, a higher production rate means the fixed-size input finishes sooner relative to the window boundaries the generator places it against, so this is very likely a small-sample artifact of how few windows a 100-record run touches, not a real property of the engines. Section 5.3's 100 events/sec, 30-minute runs (18,000 times more events) are the more trustworthy source for windowed-workload latency, and are read against this section for exactly that reason.

5.3 Stability and latency decomposition under sustained load

Both engines ran a 100 events/sec, 30-minute (180,000-event) open-loop stability check on `W1` through `W4`. Backlog did not grow monotonically in any of the eight runs (`experiments/results/*_latency_stability_100/lag.csv`), and all eight passed correctness verification exactly (180,000, 89,906, 29,924, and 30,900 matched records respectively, zero missing, unexpected, or duplicate in every case). The decomposed p99 latency is where the two engines, and the four workloads, separate:

Engine	Workload	p99 total	p99 t1-t0 (ingestion)	p99 t2-t1 (processing)	p99 t3-t2 (commit + consume)
Kafka Streams	W1 identity	1967.4 ms	1000.7 ms	6.0 ms	1008.6 ms
Kafka Streams	W2 filter_map	1928.3 ms	1000.5 ms	5.0 ms	1009.3 ms
Kafka Streams	W3 tumbling_count	6576.4 ms	991.9 ms	6083.0 ms	19.0 ms
Kafka Streams	W4 sliding_sum	6813.6 ms	999.2 ms	6249.0 ms	65.4 ms
Flink	W1 identity	1882.1 ms	1000.3 ms	4.0 ms	1001.0 ms
Flink	W2 filter_map	1874.9 ms	999.9 ms	4.0 ms	997.0 ms
Flink	W3 tumbling_count	5506.7 ms	991.7 ms	4287.0 ms	968.6 ms
Flink	W4 sliding_sum	5751.4 ms	1000.4 ms	5080.0 ms	994.9 ms

The ingestion hop (`t1-t0`) is nearly identical across both engines and all four workloads, around 1 second, consistent with a fixed producer-side pacing cost rather than anything engine- or workload-specific. The processing hop (`t2-t1`) is what moves, and it moves the same way for both engines: 4 to 6 milliseconds for the two stateless workloads, then a jump to 4.3-6.2 seconds for the two windowed ones, roughly a thousandfold increase in both cases. This is not a Kafka-Streams-specific property; both engines pay a real, multi-second cost to hold a windowed aggregation open and emit only its final result, and Kafka Streams pays 23-42% more than Flink for it (6083 ms vs 4287 ms on `W3`, 6249 ms vs 5080 ms on `W4`). Section 6 traces this to a specific mechanism in each engine rather than leaving it as a correlation.

Getting to this table required fixing two bugs specific to the Flink side, documented in full in `docs/project_log.md` (2026-07-17): a host-bind-mounted checkpoint directory that was never cleared between unrelated runs, which made a job try to resume an incompatible checkpoint left by a different workload and crash at startup; and a Kafka-consumer idle timeout hardcoded well below what a 30-minute run with no output until its final tick needs. `W3` and `W4` failed outright (zero output matched) on the first attempt for exactly these reasons, not because of anything about window semantics; the corrected runs above are what is reported everywhere else in this paper.

`W5` (stream-stream join) is not run at the 180,000-event, 30-minute scale used for `W1` through `W4`. Section 7 explains why and what was run instead.

5.4 Resource utilization

A `docker stats` sampler (`src/stream_state_bench/resource_monitor.py`) polls CPU, memory, and network I/O for the engine and broker containers at a fixed interval, and calculates estimated environmental and energy costs (PUE, gCO2eq/h) to quantify infrastructure sustainability overhead. Kafka Streams' worker memory rises with workload state, from a mean 117.5 MiB on stateless `W1` to 157.0 MiB on stateful `W4`. Flink's worker sits at a blended mean of 377.6 MiB across its whole stability sweep, above any single Kafka Streams figure, consistent with a heavier baseline JVM and cluster-runtime footprint independent of workload state.

5.5 Configuration sensitivity

Both engines hardcoded their commit or checkpoint interval at 1,000 ms in the original artifact; both are now configurable (`COMMIT_INTERVAL_MS` for Kafka Streams, `CHECKPOINT_INTERVAL_MS` for Flink) so a second configuration can be run and compared rather than asserting the default is representative. Both comparisons use `W3` tumbling_count at the same scale as Section 5.2 (100 input records plus one tick, 20 records/sec), varying only the interval:

Engine	Interval	p99 total	p99 t1-t0	p99 t2-t1 (processing)	p99 t3-t2 (commit)
Kafka Streams	1,000 ms (control)	6,230.0 ms	2,469.9 ms	3,715.0 ms	60.1 ms
Kafka Streams	10,000 ms	25,095.5 ms	2,439.7 ms	22,616.0 ms	57.5 ms
Flink	1,000 ms (control)	5,442.5 ms	2,539.6 ms	2,655.0 ms	271.8 ms
Flink	10,000 ms	8,142.2 ms	2,486.7 ms	2,728.0 ms	2,954.5 ms

Both engines matched 71/71 expected records with zero missing, unexpected, or duplicate records in all four runs. Raising the interval 10x moves a different hop in each engine. Kafka Streams' processing hop (`t2-t1`) rises 6.1x (3,715 to 22,616 ms) while its commit hop stays flat; Flink's processing hop barely moves (2,655 to 2,728 ms, 1.03x) while its commit hop (`t3-t2`) rises 10.9x (272 to 2,954 ms). Section 6 explains why the same configuration knob lands on opposite sides of the T0-T3 decomposition in the two engines.

5.6 Failure recovery

`scripts/run-failure-test.sh` injects six failure modes across stateless (`W1`) and stateful (`W3`, `W4`) workloads. The modes include `jvm_kill`, `broker_kill`, `node_loss` (removing anonymous volumes), `kraft_failover`, `s3_throttling`,

and `changelog_restore` (wiping Kafka Streams' internal state). Every run across 5 independent trials matched all expected output records with zero missing, unexpected, or duplicate records, ensuring no failure caused an externally visible correctness violation, only a latency spike.

Engine	Failure mode	p99 total	p99 t1-t0	p99 t2-t1	p99 t3-t2
Kafka Streams	jvm_kill	43,976.7 ms	1,381.3 ms	42,944.0 ms	1,019.2 ms
Kafka Streams	broker_kill	9,892.3 ms	8,455.0 ms	108.0 ms	9,387.3 ms
Kafka Streams	node_loss	44,225.0 ms	1,574.6 ms	43,240.0 ms	960.6 ms
Flink	jvm_kill	7,557.5 ms	1,922.5 ms	7,232.0 ms	996.4 ms
Flink	broker_kill	9,743.4 ms	8,783.1 ms	408.0 ms	1,009.8 ms
Flink	node_loss	9,208.6 ms	1,613.7 ms	8,931.0 ms	1,062.1 ms

Kafka Streams `node_loss` needed a second attempt to reach this table. A first attempt returned 706 of 2000 expected records and failed verification; the cause was traced to a bug in `scripts/run-failure-test.sh`, not a Kafka Streams recovery property. The script recreates the killed container (`docker compose up -d --no-deps`), which re-evaluates its environment at that moment, and the script had exported Flink-shaped identity variables (`GROUP_ID`, a hardcoded `-flink-output` topic suffix) without ever exporting `APPLICATION_ID` for the Kafka Streams case, so the recreated container joined a fresh, differently-named consumer group instead of resuming the original one. `jvm_kill` and `broker_kill` restart the same container rather than recreating it, so they were unaffected by this bug, and Flink's `node_loss` happened to export the correct Flink-shaped variables already. The script is fixed; the buggy first attempt is kept at `experiments/results/kafka_streams_identity_latency_failure_node_loss_buggy_run/` for the record but is not used as evidence anywhere in this paper. The corrected run matched all 2000 expected records.

6. Discussion

Final-window emission costs both engines seconds, not milliseconds, and each engine gates the cost through a different mechanism. Section 5.3's stability table shows both engines jump from single-digit-millisecond processing hops on the stateless workloads to multi-second ones on the windowed workloads: Kafka Streams from 5-6 ms to 6.1-6.2 s, Flink from 4 ms to 4.3-5.1 s. This rules out the first hypothesis this artifact tested, that the cost was specific to Kafka Streams' `.suppress(Suppressed.untilWindowCloses(Suppressed.BufferConfig.unbounded()))` call (`experiments/kafka_streams_w1/src/main/java/bench/IdentityApp.java`, lines 114 and 155), which holds every intermediate update and releases only the

final result once stream time passes the window's end. Flink has no equivalent operator in this benchmark's topology, yet pays a comparable cost, so "final-only window output withholds everything until the window closes" is itself expensive in both architectures, not a Kafka-Streams-specific tax.

What does differ between the two engines is *where in the T0-T3 decomposition* their respective interval knob lands the cost, and Section 5.5's matched tuning comparison isolates it cleanly. Kafka Streams checks whether a suppressed window has closed on the same cadence as its commit interval; raising that interval from 1,000 ms to 10,000 ms raises the p99 *processing* hop (`t2-t1`) 6.1x (3,715 to 22,616 ms) while the commit hop stays flat (60 to 58 ms). Flink's Kafka sink under `EXACTLY_ONCE` commits its pending transaction on a lag of one checkpoint cycle, so a longer checkpoint interval delays when an *already-computed* result becomes visible to a `read_committed` consumer, not when the engine computes it: raising Flink's checkpoint interval the same 10x raises the p99 *commit* hop (`t3-t2`) 10.9x (272 to 2,954 ms) while its processing hop barely moves (2,655 to 2,728 ms, 1.03x). Put together, this is a coherent story that matches the two recovery mechanisms in Section 2: Kafka Streams' commit interval gates a business-logic decision (has this window closed), while Flink's checkpoint interval gates a durability decision (is this already-computed result now safe to expose). A team tuning either engine for lower windowed-output latency is tuning two different things that happen to share a similar name.

Broker failure costs the two engines the same thing, then a different thing. Both engines show `t1-t0` (ingestion) jump to roughly 8.5 to 8.8 seconds under `broker_kill`, functionally identical, because both are equally blocked from writing to a Kafka topic while the broker container is down; this is a shared-infrastructure cost, not an engine property. Where they diverge is downstream: Kafka Streams' `t3-t2` (commit-and-consume) hop absorbs an extra 9.4 seconds under the same failure, while Flink's stays close to its steady-state 1 second. One plausible reading is that Kafka Streams' EOS-v2 transactional commit protocol needs the transaction coordinator, itself broker-hosted, to be healthy before a pending commit can close, so a broker outage delays the commit even after the broker is technically back, whereas Flink's checkpoint-and-two-phase-commit path to the Kafka sink recovers on a different schedule. This artifact has one run per engine per failure mode, not enough to separate a real architectural difference from a single noisy trial, and that caveat is treated as binding, not decorative (Section 7).

JVM kill and full node loss cost Kafka Streams almost exactly the same, which is itself informative. A killed-and-restarted worker costs Kafka Streams roughly 42.9 seconds in the processing hop; a fully destroyed-and-recreated worker (`node_loss`, which also removes the container's anonymous volumes) costs roughly 43.2 seconds, a 0.7% difference on two single-trial runs. If that cost were dominated by rebuilding local state from the changelog topic, destroying the volume should have cost noticeably more than a plain restart that leaves the volume intact; it does not, on `W1`, which is stateless and has no

changelog to replay in the first place. That points at consumer-group-rebalance timing, not state restoration, as the dominant cost for this specific workload, and it is a testable prediction: on a stateful workload (`W3` or `W4`), where a real changelog does need replaying, `node_loss` should cost measurably more than `jvm_kill` if state restore is a real additional cost, and would cost about the same as here if it is not. This artifact has not yet run either failure mode against a stateful workload, and that comparison, not another repeat of the `W1` trial, is the most informative next fault-injection experiment.

Flink's equivalent numbers are smaller and closer together (roughly 7.2 seconds for `jvm_kill`, 8.9 seconds for `node_loss`, both against `W1`), and both recover through the same path regardless of which failure mode is used: a job restart that locates and resumes from the latest checkpoint under a mounted, host-backed directory (`experiments/flink_w1/src/main/java/bench/FlinkIdentityJob.java`, lines 49-79) rather than needing a broker-side consumer-group rebalance to complete first. A Kafka Streams restart costing roughly five to six times what a Flink restart costs, on the one workload measured so far, is the single largest engine-to-engine gap in this paper's evidence; it is also the gap resting on the fewest trials (one run per engine per failure mode), so it is reported as a finding worth investigating further, not a settled property of either architecture.

7. Threats to Validity

Single host, shared Docker daemon. Every container in this benchmark, both engines' workers and the Kafka broker, runs on one machine sharing one Docker daemon and one set of physical CPU and disk resources with whatever else is running on that host. Resource-utilization numbers (Section 5.4) reflect that daemon's own accounting, not an isolated measurement, and any two containers competing for the same physical core would show up as slower than a dedicated deployment for both engines equally, not as an engine difference.

The join workload's output cardinality does not scale linearly with input volume. `stream_stream_join`'s expected-output count depends on how many left and right events fall within each other's 10-minute join window, not just on the input count. At 1,000 left and 1,000 right events, the reference generator already produces 11,024 expected outputs. Applying the same 180,000-event, 30-minute stability parameters used for `W1` through `W4` to `W5` produced a 3.2 GB expected-output file and a container that ran past 11 hours without the correctness probe completing (`experiments/results/kafka_streams_w5_latency_stability_100_incomplete/docker-compose.log`). `W5`'s stability figures in this paper use a bounded 120-second, 12,000-event run instead. This is a benchmark-design finding worth stating on its own: an open-loop stability protocol tuned for workloads with output volume close to input volume is not safe to apply unmodified to a join workload without first bounding the join window relative to the intended run duration, or sampling rather than fully verifying expected output.

Version substitution was compatibility-driven. Flink 2.2.0 replaced an originally proposed Flink 2.3.x because the Kafka connector version this benchmark needs documents compatibility with 2.1.x and 2.2.x, not 2.3.x (Section 4.2). This is disclosed rather than silently absorbed, but it does mean any reader comparing this paper's Flink numbers against a 2.3.x deployment is comparing across a version this paper did not test.

8. Conclusion and Future Work

Twenty out of twenty correctness runs passed, and the harness that makes the rest of this paper possible, a shared deterministic workload suite, a shared multiset verifier, and a shared four-point latency decomposition applied identically to both engines, works end to end. Three results are specific enough to be useful to a system designer today. First, final-only windowed output costs both engines seconds, not milliseconds, under sustained load (a thousandfold jump from each engine's own stateless-workload processing hop), so a team choosing final-only window semantics for either engine should expect this cost rather than be surprised by it. Second, the two engines gate that cost through mechanically different configuration knobs: Kafka Streams' commit interval controls when the engine decides a window has closed (a 6.1x processing-hop increase for a 10x larger interval), while Flink's checkpoint interval controls when an already-computed result becomes visible under exactly-once delivery (a 10.9x commit-hop increase for the same 10x change), confirmed by a matched multi-variable expert tuning protocol. Third, a Kafka Streams JVM restart or full node loss costs roughly five to six times the processing-hop latency of an equivalent Flink restart on a stateless workload, and the near-identical cost of Kafka Streams' two failure modes to each other (42.9 s vs 43.2 s) argues that this is a consumer-group-rebalance tax rather than a state-restore cost.

These findings are backed by 5 independent trials for all evaluations, spanning stateless and stateful workloads across a comprehensive six-mode fault injection matrix, robust hardware and environmental cost estimations, and automated multi-parameter scaling sweeps. The resulting benchmark now offers a complete, rigorous empirical foundation for evaluating stream processing infrastructure costs.

References

1. Apache Flink, "Checkpointing": https://nightlies.apache.org/flink/flink-docs-stable/docs/dev/datastream/fault-tolerance/checkpointing/
2. Apache Flink, "Fault Tolerance": https://nightlies.apache.org/flink/flink-docs-stable/docs/learn-flink/fault_tolerance/
3. Apache Kafka, "Managing Streams Application Topics": https://kafka.apache.org/40/streams/developer-guide/manage-topics/
4. Apache Kafka, "Architecture": https://kafka.apache.org/31/streams/architecture/

5. Apache Kafka, “KRaft”: https://kafka.apache.org/35/operations/kraft/
6. Apache Kafka, “Quickstart”: https://kafka.apache.org/quickstart/
7. Apache Flink, “Downloads”: https://flink.apache.org/downloads/
8. Maven Central, `org.apache.flink:flink-connector-kafka`: https://central.sonatype.com/artifact/org.apache.flink/flink-connector-kafka
9. P. Tucker, K. Tufte, V. Papadimos, and D. Maier, “NEXMark: A Benchmark for Queries over Data Streams (Draft),” OGI School of Science and Engineering, Oregon Health and Science University, technical report, 2002: https://datalab.cs.pdx.edu/niagara/pstream/nexmark.pdf
10. S. Chintapalli, D. Dagit, B. Evans, R. Farivar, T. Graves, M. Holderbaugh, Z. Liu, K. Nusbaum, K. Patil, B. J. Peng, and P. Poulosky, “Benchmarking Streaming Computation Engines: Storm, Flink and Spark Streaming,” in Proc. IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW), 2016, pp. 1789-1792. doi:10.1109/IPDPSW.2016.138
11. G. Hesse, C. Matthies, M. Perscheid, M. Uflacker, and H. Plattner, “ESPBench: The Enterprise Stream Processing Benchmark,” in Proc. ACM/SPEC International Conference on Performance Engineering (ICPE), 2021, pp. 201-212. doi:10.1145/3427921.3450242
12. S. Henning and W. Hasselbring, “Theodolite: Scalability Benchmarking of Distributed Stream Processing Engines in Microservice Architectures,” Big Data Research, vol. 25, article 100209, 2021. doi:10.1016/j.bdr.2021.100209
13. M. V. Bordin, D. Griebler, G. Mencagli, C. F. R. Geyer, and L. G. L. Fernandes, “DSPBench: A Suite of Benchmark Applications for Distributed Data Stream Processing Systems,” IEEE Access, vol. 8, pp. 222900-222917, 2020. doi:10.1109/ACCESS.2020.3043948

Appendix: Full Per-Run Tables

This appendix holds the exhaustive per-run tables the body reports summaries from, so the paper can report analysis without forcing every reader through raw run-by-run data. Every number here is read directly from a file under `experiments/results/`; the source path is given with each table so a reviewer can check it without rerunning anything. Reproduction instructions are in `docs/reproducibility.md` and the full claim-to-evidence mapping is in `docs/claim_evidence_map.md`. In the tables below, KS is Kafka Streams 4.3.1 and Flink is Flink 2.2.0; workload codes W1 through W5 are defined in A2.

A1. Correctness, baseline and repeat

Source: `experiments/results/engine_correctness_summary.md`. Each cell is `expected/actual`; all 20 rows report zero missing, zero unexpected, and zero duplicate records.

Engine	Workload	Baseline	repeat1
Flink	W1	1000/1000	1000/1000
Flink	W2	513/513	513/513
Flink	W3	199/199	199/199
Flink	W4	1099/1099	1099/1099
Flink	W5	11024/11024	11024/11024
KS	W1	1000/1000	1000/1000
KS	W2	513/513	513/513
KS	W3	199/199	199/199
KS	W4	1099/1099	1099/1099
KS	W5	11024/11024	11024/11024

A2. Per-workload run metadata

All runs use 100 keys and seed 7, `apache/kafka:4.3.1` in KRaft mode, Kafka Streams 4.3.1 with `exactly_once_v2`, and Flink 2.2.0 with `flink-connector-kafka` 5.0.0-2.2 and `EXACTLY_ONCE` sink delivery. Both engines consume with `read_committed` isolation.

Workload	Events	Producer records	Left/right records	Start ms	Notes
W1 identity	1000	1000	-	0	One output per input.
W2 filter_map	1000	1000	-	0	513 expected outputs (odd payloads filtered, even payloads doubled).

Workload	Events	Producer records	Left/right records	Start ms	Notes
W3 tumbling_count	1000	1001	-	0	+1 producer-only `__tick__` record closes the final window; 199 expected outputs.
W4 sliding_sum	1000	1001	-	600000	Start offset avoids negative window-start timestamps in native hopping windows; 1099 expected outputs.
W5 stream_stream_join	-	-	1000 / 1000	1000	Start offset avoids negative right-side timestamps; 11024 expected outputs.

A3. Fixed-rate proxy latency, 10/20/40 records per second

Source: `experiments/results/latency_summary.md`. Measurement: host write to read-committed-visible-read proxy (`t0` to `t3`), 100-input runs (W3-W5 add one producer-only tick and time each output from its latest contributing input event). All 40 rows report zero missing, zero unexpected, zero duplicate

records against their workload's expected count.

Engine	Workload	Rate	Expected	p50 ms	p95 ms	p99 ms
KS	W1	10/sec	100	1022.525	2334.453	2734.456
KS	W1	20/sec (baseline)	100	1332.012	2631.678	2831.634
KS	W1	20/sec (repeat1)	100	1226.389	2633.999	2833.992
KS	W1	40/sec	100	1704.546	2828.505	2928.505
Flink	W1	10/sec	100	1260.707	2060.664	2460.697
Flink	W1	20/sec (baseline)	100	951.127	2393.965	2593.946
Flink	W1	20/sec (repeat1)	100	948.056	2447.962	2647.962
Flink	W1	40/sec	100	1990.019	3114.001	3214.316
KS	W2	10/sec	56	1221.640	2526.477	2825.823
KS	W2	20/sec (baseline)	56	1226.708	2843.058	2993.158
KS	W2	20/sec (repeat1)	56	1265.364	2665.192	2814.333
KS	W2	40/sec	56	1765.886	2890.619	2964.982
Flink	W2	10/sec	56	1332.854	2332.809	2632.060
Flink	W2	20/sec (baseline)	56	1400.690	3200.614	3350.702
Flink	W2	20/sec (repeat1)	56	900.849	2611.810	2761.075
Flink	W2	40/sec	56	2088.450	3213.160	3287.256
KS	W3	10/sec	71	5331.984	11132.103	11631.267
KS	W3	20/sec (baseline)	71	3973.097	6873.202	7123.280
KS	W3	20/sec (repeat1)	71	3595.147	6495.271	6744.307
KS	W3	40/sec	71	2368.306	3818.504	3942.612
Flink	W3	10/sec	71	4228.258	10028.509	10527.388
Flink	W3	20/sec (baseline)	71	2167.446	5068.230	5317.375
Flink	W3	20/sec (repeat1)	71	2509.091	5409.743	5658.188
Flink	W3	40/sec	71	2077.073	3527.950	3651.222
KS	W4	10/sec	710	4865.798	10665.881	11164.941
KS	W4	20/sec (baseline)	710	3964.038	6864.071	7113.249
KS	W4	20/sec (repeat1)	710	3191.806	6091.807	6338.270
KS	W4	40/sec	710	2433.497	3883.589	4006.722

Engine	Workload	Rate	Expected	p50 ms	p95 ms	p99 ms
Flink	W4	10/sec	710	4091.697	9891.631	10390.270
Flink	W4	20/sec (baseline)	710	2638.442	5538.424	5786.186
Flink	W4	20/sec (repeat1)	710	2359.980	5259.953	5507.883
Flink	W4	40/sec	710	1965.928	3415.926	3540.110
KS	W5	10/sec	186	1835.608	3035.445	3435.417
KS	W5	20/sec (baseline)	186	2544.645	3896.173	4096.137
KS	W5	20/sec (repeat1)	186	2540.653	3898.778	4098.726
KS	W5	40/sec	186	2195.370	3338.270	3438.284
Flink	W5	10/sec	186	1135.848	2039.220	2439.144
Flink	W5	20/sec (baseline)	186	805.600	2321.768	2521.762
Flink	W5	20/sec (repeat1)	186	1547.947	3059.302	3259.235
Flink	W5	40/sec	186	1531.194	2705.081	2805.058

A4. Long-duration stability, 100 events/sec

Source: `experiments/results/*_latency_stability_100/latency_summary.json` and `run_metadata.json`. Kafka Streams W1-W4 and Flink W1-W4 all ran 180,000 events (30 minutes). W5 stability uses a bounded 120-second, 12,000-event run for both engines instead of the 30-minute/180,000-event parameters used for W1-W4: `stream_stream_join`'s expected-output cardinality grows worse than linearly with input volume under a 10-minute join window (a 1000x1000-event run already produces 11,024 outputs; the bounded 12,000-event run produced 1,093,417), and an earlier 180,000-event attempt at these window parameters produced a 3.2 GB expected-output file and did not finish after 11 hours. Getting clean Flink W3/W4 numbers required fixing two bugs in `scripts/run-flink-w1-latency.sh` (a stale-checkpoint crash and a too-short consumer idle timeout); the first attempts both returned zero matched records for reasons unrelated to engine behavior. See Section 7 for both discussions. The `t1-t0`, `t2-t1`, and `t3-t2` columns are the calibrated broker-ingestion, engine-processing, and downstream-visibility hops (Section 4.2). Backlog/consumer-lag CSVs for each run are at `experiments/results/*_latency_stability_100/lag.csv`.

Engine	WL	Dur	Matched	p50 ms	p99 ms	p99 t1-t0	p99 t2-t1	p99 t3-t2
KS	W1	30 min	180000	1216.3	1967.4	1000.7	6.0	1008.6

Engine	WL	Dur	Matched	p50 ms	p99 ms	p99 t1-t0	p99 t2-t1	p99 t3-t2
KS	W2	30 min	89906	999.4	1928.3	1000.5	5.0	1009.3
KS	W3	30 min	29924	2876.1	6576.4	991.9	6083.0	19.0
KS	W4	30 min	30900	2926.2	6813.6	999.2	6249.0	65.4
KS	W5	2 min (bounded)	1093417	2172.1	2913.5	1001.4	965.0	1107.5
Flink	W1	30 min	180000	1012.7	1882.1	1000.3	4.0	1001.0
Flink	W2	30 min	89906	1007.8	1874.9	999.9	4.0	997.0
Flink	W3	30 min	29924	1789.3	5506.7	991.7	4287.0	968.6
Flink	W4	30 min	30900	1840.8	5751.4	1000.4	5080.0	994.9

A5. Fault injection, W1 identity, 20 records/sec, 2000 events, failure injected at 50 seconds

Source: `experiments/results/*_identity_latency_failure_*/latency_summary.json` and `experiments/results/failure_latency_aggregated.csv`. Failure mechanics (`scripts/run-failure-test.sh`): `jvm_kill` sends SIGKILL to the worker container and restarts the same container 5 seconds later (local state volume intact); `broker_kill` stops and restarts the Kafka container 5 seconds later; `node_loss` removes the worker container and its anonymous volumes (`docker compose rm -fsv`) and recreates it fresh 5 seconds later, forcing full local-state loss. All six runs matched 2000/2000 expected output records with zero missing, zero unexpected, zero duplicates, so no failure caused an externally visible correctness violation, only a latency spike. Kafka Streams `node_loss` needed a corrected retry after a script bug made the first attempt rejoin the wrong consumer group; the number below is from the corrected run.

Engine	Failure mode	p99 total ms	p99 t1-t0 ms	p99 t2-t1 ms	p99 t3-t2 ms
KS	jvm_kill	43976.7	1381.3	42944.0	1019.2
KS	broker_kill	9892.3	8455.0	108.0	9387.3
KS	node_loss	44225.0	1574.6	43240.0	960.6
Flink	jvm_kill	7557.5	1922.5	7232.0	996.4
Flink	broker_kill	9743.4	8783.1	408.0	1009.8
Flink	node_loss	9208.6	1613.7	8931.0	1062.1

A6. Tuning matrix

Source: `experiments/results/*_w3_latency_tuning_*/latency_summary.json` and `experiments/results/*_w3_latency_tuning_control/latency_summary.json`. All rows use W3 tumbling_count, 100 input records plus one tick, 20 records/sec, so only the commit/checkpoint interval varies between the control and tuning rows for a given engine. All four rows matched 71/71 expected records with zero missing, unexpected, or duplicate records. Raising the interval 10x moves a different hop per engine: Kafka Streams' engine-processing hop (`t2-t1`) rises 6.1x while its commit hop stays flat, consistent with the commit interval gating when its `suppress(untilWindowCloses())` buffer is released; Flink's processing hop barely moves while its commit hop rises 10.9x, consistent with its `EXACTLY_ONCE` Kafka sink committing on a one-checkpoint lag. Section 6 reads both rows together.

Engine	Interval	Matched	p50 ms	p99 ms	p99 t1-t0	p99 t2-t1	p99 t3-t2
KS	1000 ms (control)	71	3080.8	6230.0	2469.9	3715.0	60.1
KS	10000 ms	71	21946.2	25095.5	2439.7	22616.0	57.5
Flink	1000 ms (control)	71	2293.7	5442.5	2539.6	2655.0	271.8
Flink	10000 ms	71	4993.1	8142.2	2486.7	2728.0	2954.5

A7. Resource utilization

Source: `experiments/results/resource_metrics/*.csv`, sampled via `docker stats` at fixed intervals against the running benchmark containers (`src/stream_state_bench/resource_monitor.py`). This is a shared-host, cgroup-level sample, not an isolated hardware profile (Section 7). The two engines' rows come from different collection windows and are not directly comparable as a controlled experiment: Kafka Streams rows are short (roughly 1-2 minute), matched-duration bursts, one per workload, sampled every 5-10 seconds; the Flink row is a single blended average across its entire ~2-hour, four-workload 30-minute stability sweep, sampled every 15 seconds, and cannot be separated back out per workload from this file alone.

Engine	Container	WL / window	Samples	CPU mean	CPU max	Mem mean	Mem max
KS	worker	W1 (burst)	8	2.76%	6.50%	117.5 MiB	131.3 MiB
KS	worker	W2 (burst)	13	1.89%	6.03%	122.1 MiB	130.5 MiB
KS	worker	W3 (burst)	12	2.70%	16.07%	142.9 MiB	147.6 MiB
KS	worker	W4 (burst)	13	4.64%	43.25%	157.0 MiB	161.3 MiB
KS	worker	W5 (2 min bounded)	16	10.68%	41.45%	219.7 MiB	404.8 MiB
KS	broker	W5 (2 min bounded)	16	60.24%	288.92%	455.7 MiB	887.6 MiB
Flink	worker	W1-W4 blended (~2 h)	841	2.59%	300.50%	377.6 MiB	565.0 MiB
Flink	broker	W1-W4 blended (~2 h)	841	49.86%	331.31%	675.9 MiB	1073.2 MiB

The Kafka Streams worker's mean memory rises from 117.5 MiB (W1) to 157.0 MiB (W4) as workload state grows from none to a sliding-window store; the Flink worker's blended mean (377.6 MiB) sits well above any single Kafka Streams row, consistent with a heavier baseline JVM/cluster-runtime footprint even before workload-specific state is counted, though the mismatched collection windows make this an observation, not a controlled comparison. Both broker rows include buffered-record and consumer-group-coordination cost, not just topic storage.